



3η Εργαστηριακή Άσκηση

Ψηφιακά Συστήματα VLSI

Παναγιώτης Γερασιμόπουλος 03115208
personal@devcol.com

Contents

1. Ram και Rom	1
2. Multiplier Accumulator	2
3. Control Unit	3
4. FIR	4
4.1. Implementation	4
4.2. Test Bench	7

1. Ram και Rom

Τροποποιήθηκαν τα αρχεία για τις μνήμες ώστε να μπορούμε με generics να καθορίσουμε όλα τα μεγέθη στο instantiation σε περίπτωση που χρειαστεί σε επόμενες ασκήσεις, η λειτουργία παραμένει όμως ίδια για την Rom. Για την Ram προστέθηκε άπλα ένα Reset, ένα for loop στο write enable για την ολίσθηση και μετά μια φόρτωση στο index 0. Δεν ήταν ξεκάθαρο από την εκφώνηση τι πρέπει να είναι το output μετά από ένα write οπότε άπλα δίνει ότι διεύθυνση παίρνει στην είσοδο.

```
1  -- =====
2  --                               Package boilerplate
3  -- =====
4
5  library ieee;
6  use ieee.std_logic_1164.all;
7  use ieee.math_real.all;
8
9  -- NOTE(acol): ugly boilerplate so I can pass generics on instantiation
10 package Memories is
11     type reg_arr is array (natural range <=>) of std_logic_vector;
12
13     -- NOTE(acol): just log2 followed by a ceiling op
14     function Clog2(Val : positive) return natural;
15 end package Memories;
16
17 package body Memories is
18     function Clog2(Val : positive) return natural is
19     begin
20         -- anti retard shielding
21         if Val <= 1 then
22             return 1;
23         else
24             return integer(ceil(log2(real(Val))));
25         end if;
26     end function;
27 end package body Memories;
28
29 -- =====
30 --                               Rom implementation
31 -- =====
32
33 library ieee;
34 use ieee.std_logic_1164.all;
35 use ieee.numeric_std.all;
36 use work.Memories.all;
37
38 entity Rom is
39     generic(
40         BIT_COUNT : integer;
41         REG_COUNT : integer;
42         COEFFS     : reg_arr
43     );
44     Port(
45         Clk, Enable : in std_logic;
46         Address      : in std_logic_vector(Clog2(REG_COUNT) - 1 downto 0);
47         RomOut       : out std_logic_vector(BIT_COUNT-1 downto 0)
48     );
49 end entity;
50
51 architecture Behavioral of Rom is
52 begin
53     process(Clk) begin
54         if rising_edge(Clk) then
55             if (Enable = '1') then
56                 RomOut <= COEFFS(to_integer(unsigned(Address)));
```

```
57     end if;
58     end if;
59 end process;
60 end architecture;
61
62 -- =====
63 --                      Ram implementation
64 -- =====
65
66 library ieee;
67 use ieee.std_logic_1164.all;
68 use ieee.numeric_std.all;
69 use work.Memories.all;
70
71 entity Ram is
72     generic(
73         BIT_COUNT : integer;
74         REG_COUNT : integer
75     );
76     Port(
77         Clk, Enable, WEnable, Reset : in std_logic;
78         Address                     : in std_logic_vector(Clog2(REG_COUNT) - 1 downto 0);
79         DataIn                      : in std_logic_vector(BIT_COUNT - 1 downto 0);
80         DataOut                     : out std_logic_vector(BIT_COUNT - 1 downto 0)
81     );
82 end entity;
83
84 architecture Behavioral of Ram is
85     type ram_type is array (0 to REG_COUNT-1) of std_logic_vector(BIT_COUNT-1 downto 0);
86
87     signal RamEntries : ram_type := (others => (others => '0'));
88 begin
89     process(Clk, Reset) begin
90         if Reset = '1' then
91             RamEntries <= (others => (others => '0'));
92             DataOut    <= (others => '0');
93
94         elsif rising_edge(Clk) then
95
96             if Enable = '1' then
97                 if WEnable = '1' then
98                     for i in 0 to REG_COUNT - 2 loop
99                         RamEntries(i + 1) <= RamEntries(i);
100                     end loop;
101                     RamEntries(0) <= DataIn;
102
103                     DataOut <= DataIn;
104                 else
105                     DataOut <= RamEntries(to_integer(unsigned(Address)));
106                 end if;
107             end if;
108
109         end if;
110     end process;
111 end architecture;
112
```

2. Multiplier Accumulator

Για τον αριθμό bit του MAC ισχύει ότι

$$L(y) = 2N + \log_2(M + 1) \quad (2.1)$$

άρα θέλουμε 19 bit. Έκτος αυτού η δομή του είναι προφανής από τον κώδικα.

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity Mac is
6   Port(
7     Clk, Enable, Reset, Init : in std_logic;
8     IRCoeff, Sig : in std_logic_vector(7 downto 0);
9     Y0out : out std_logic_vector(18 downto 0)
10  );
11 end entity;
12
13 architecture Behavioral of Mac is
14   signal Acc : std_logic_vector(18 downto 0);
15 begin
16
17   process(Clk, Reset) begin
18     if Reset = '1' then
19       Acc <= (others => '0');
20
21     elsif rising_edge(Clk) then
22       if Init = '1' then
23         -- NOTE(acol): NOT 0 casue you miss a tap... :(
24         Acc <= "000" & (IRCoeff * Sig);
25       elsif Enable = '1' then
26         Acc <= Acc + ("000" & (IRCoeff * Sig));
27       end if;
28     end if;
29   end process;
30
31   Y0out <= Acc;
32 end architecture;
33
```

3. Control Unit

Το μόνο που ίσως να αξίζει σχολιασμό είναι ότι υπερिशύει το τελευταίο assignment οπότε μπορούμε να κάνουμε πάντα αρχικοποίηση των σημάτων σε default και μετά να τα αλλάζουμε, αλλιώς το κύκλωμα είναι πολύ απλό.

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity CU is
6   Port(
7     Clk, Reset, ValidIn      : in std_logic;
8     ValidOut, RomEnable, RamEnable : out std_logic;
9     RamWEnable, MacEnable, MacInit : out std_logic;
10    RamAddress, RomAddress      : out std_logic_vector(2 downto 0)
11  );
12 end entity;
13
14 architecture Behavioral of CU is
15   signal Counter : std_logic_vector(2 downto 0) := (others => '0');
16 begin
17
18   process(Clk, Reset) begin
19     if Reset = '1' then
20       Counter <= (others => '0');
21       ValidOut <= '0';
22       --RomEnable <= '0';
23       RamEnable <= '0';
24       RamWEnable <= '0';

```

```

25     MacEnable <= '0';
26     MacInit <= '0';
27
28     elsif rising_edge(Clk) then
29
30         RomEnable <= '1';
31         RamEnable <= '1';
32         MacEnable <= '1';
33         MacInit <= '0';
34         ValidOut <= '0';
35         RamWEnable <= '0';
36
37         if Counter = "111" then
38             ValidOut <= '1';
39         end if;
40
41         if ValidIn = '1' then
42             RamWEnable <= '1';
43             MacInit <= '1';
44             Counter <= "000";
45         else
46             Counter <= Counter + 1;
47         end if;
48     end if;
49 end process;
50
51 RamAddress <= Counter;
52 RomAddress <= Counter;
53 end architecture;
54
55

```

4. FIR

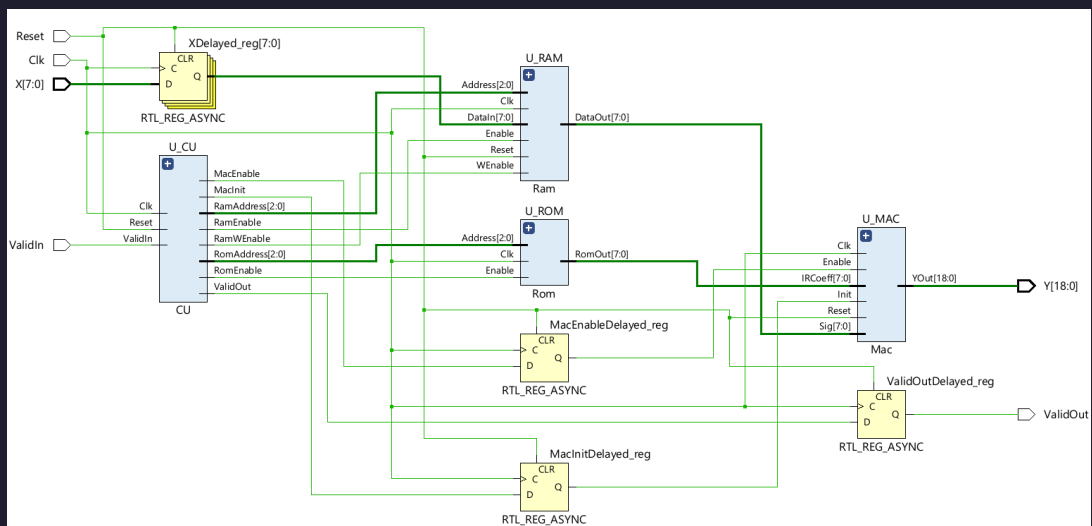


Figure 2: RTL

4.1. Implementation

Για το τελικό σχέδιο συνδέουμε όλα τα παραπάνω components και προσθέτουμε καθυστέρηση στο X μέχρι την ram και σε όλα τα σήματα από την cu στον mac.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use work.Memories.all;
4
5  entity FirFilter is
6      Port(
7          Clk          : in std_logic;
8          Reset        : in std_logic;
9          ValidIn       : in std_logic;
10         X             : in std_logic_vector(7 downto 0);
11         ValidOut      : out std_logic;
12         Y             : out std_logic_vector(18 downto 0)
13     );
14 end entity;
15
16 architecture Structural of FirFilter is
17
18     signal XDelayed      : std_logic_vector(7 downto 0);
19     signal MacInitDelayed : std_logic;
20     signal MacEnableDelayed : std_logic;
21     signal ValidOutDelayed : std_logic;
22
23     signal CuValidOut, CuRomEnable, CuRamEnable, CuRamWEnable : std_logic;
24     signal CuMacEnable, CuMacInit                             : std_logic;
25     signal CuRamAddress, CuRomAddress                         : std_logic_vector(2 downto 0);
26
27     signal RomDataOut, RamDataOut                             : std_logic_vector(7 downto 0);
28
29 begin
30
31     U_CU: entity work.CU
32     port map(
33         Clk      => Clk,
34         Reset    => Reset,
35         ValidIn  => ValidIn,
36         ValidOut => CuValidOut,
37         RomEnable => CuRomEnable,
38         RamEnable => CuRamEnable,
39         RamWEnable => CuRamWEnable,
40         MacEnable => CuMacEnable,
41         MacInit   => CuMacInit,
42         RamAddress => CuRamAddress,
43         RomAddress => CuRomAddress
44     );
45
46     U_ROM: entity work.Rom
47     generic map(
48         BIT_COUNT => 8,
49         REG_COUNT => 8,
50         COEFFS    => ("00000001", "00000010", "00000011", "00000100",
51                     "00000101", "00000110", "00000111", "00001000")
52     )
53     port map(
54         Clk      => Clk,
55         Enable   => CuRomEnable,
56         Address  => CuRomAddress,
57         RomOut   => RomDataOut
58     );
59
60     U_RAM: entity work.Ram
61     generic map(
62         BIT_COUNT => 8,
63         REG_COUNT => 8
64     )
65     port map(
66         Clk      => Clk,
67         Reset    => Reset,
68         Enable   => CuRamEnable,
69         WEnable  => CuRamWEnable,
70         Address  => CuRamAddress,

```

```

71     DataIn  => XDelayed,
72     DataOut => RamDataOut
73 );
74
75 U_MAC: entity work.Mac
76 port map(
77     Clk      => Clk,
78     Reset    => Reset,
79     Enable   => MacEnableDelayed,
80     Init     => MacInitDelayed,
81     IRCoeff  => RomDataOut,
82     Sig      => RamDataOut,
83     Y0out    => Y
84 );
85
86 process(Clk, Reset) begin
87     if Reset = '1' then
88         XDelayed      <= (others => '0');
89         MacInitDelayed <= '0';
90         MacEnableDelayed <= '0';
91         ValidOutDelayed <= '0';
92     elsif rising_edge(Clk) then
93         XDelayed      <= X;
94         MacInitDelayed <= CuMacInit;
95         MacEnableDelayed <= CuMacEnable;
96         ValidOutDelayed <= CuValidOut;
97     end if;
98 end process;
99
100 ValidOut <= ValidOutDelayed;
101 end architecture;
102

```

Τελικά, όλο αυτό μας δίνει στο implementation critical path 6.975ns και άρα συχνότητα $f \approx 143$ MHz

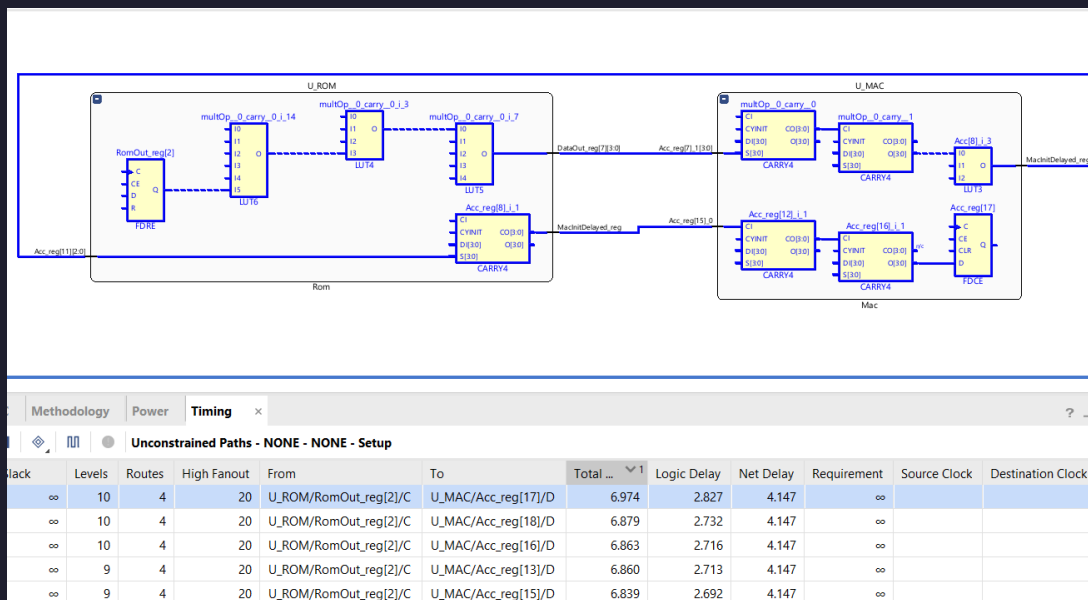


Figure 3: Critical path 6.975

και utilization

Name	^1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Bonded IOB (100)	BUFGCTRL (32)
▼ N FirFilter		83	112	39	83	31	1
U_CU (CU)		5	6	5	5	0	0
U_MAC (Mac)		19	19	9	19	0	0
U_RAM (Ram)		34	72	25	34	0	0
U_ROM (Rom)		27	4	15	27	0	0

Name	^1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Bonded IOB (100)	BUFGCTRL (32)
▼ N FirFilter		0.47%	0.32%	0.89%	0.47%	31.00%	3.13%
U_CU (CU)		0.03%	0.02%	0.11%	0.03%	0.00%	0.00%
U_MAC (Mac)		0.11%	0.05%	0.20%	0.11%	0.00%	0.00%
U_RAM (Ram)		0.19%	0.20%	0.57%	0.19%	0.00%	0.00%
U_ROM (Rom)		0.15%	0.01%	0.34%	0.15%	0.00%	0.00%

4.2. Test Bench

Για δοκιμές επιλέχθηκαν δυο σήματα, μια Dirac και ένα unit step.

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity TbFirFilter is
5 end entity;
6
7 architecture Simulation of TbFirFilter is
8     signal Clk      : std_logic := '0';
9     signal Reset    : std_logic := '0';
10    signal ValidIn   : std_logic := '0';
11    signal X         : std_logic_vector(7 downto 0) := (others => '0');
12    signal ValidOut  : std_logic;
13    signal Y         : std_logic_vector(18 downto 0);
14
15    constant CLK_PERIOD : time := 10 ns;
16 begin
17     UUT: entity work.FirFilter
18     port map(
19         Clk      => Clk,
20         Reset    => Reset,
21         ValidIn  => ValidIn,
22         X        => X,
23         ValidOut => ValidOut,
24         Y        => Y
25     );
26
27     Clk <= not Clk after CLK_PERIOD / 2;
28
29     process begin
30         Reset <= '1';
31         wait for CLK_PERIOD * 2;
32         Reset <= '0';
33         -- Dirac
34         for i in 0 to 9 loop
35             ValidIn <= '1';
36             if i = 0 then
37                 X <= "00000001";
38             end if;
39             wait for CLK_PERIOD;
40             X <= "00000000";
41             ValidIn <= '0';

```

```

42     wait for CLK_PERIOD * 7;
43   end loop;
44   -- flush
45
46   Reset <= '1';
47   wait for CLK_PERIOD * 2;
48   Reset <= '0';
49   -- Step
50   for i in 0 to 9 loop
51     ValidIn <= '1';
52     X <= "00000001";
53     wait for CLK_PERIOD;
54     ValidIn <= '0';
55     X <= "00000000";
56     wait for CLK_PERIOD * 7;
57   end loop;
58   -- flush
59   wait for CLK_PERIOD * 20;
60
61 end process;
62 end architecture;
63
64

```

Περιμένουμε να δούμε κατά το πρώτο σήμα τα coefficients και κατά το δεύτερο περιμένουμε να δούμε ένα accumulation των τιμών του προηγούμενου. Δηλαδή

$$y_1[n] = \left\{ \underset{\uparrow}{1}, 2, 3, 4, 5, 6, 7, 8 \right\} \quad (4.1)$$

$$y_2[n] = \left\{ \underset{\uparrow}{1}, 3, 6, 10, 15, 21, 28, 36 \right\}$$

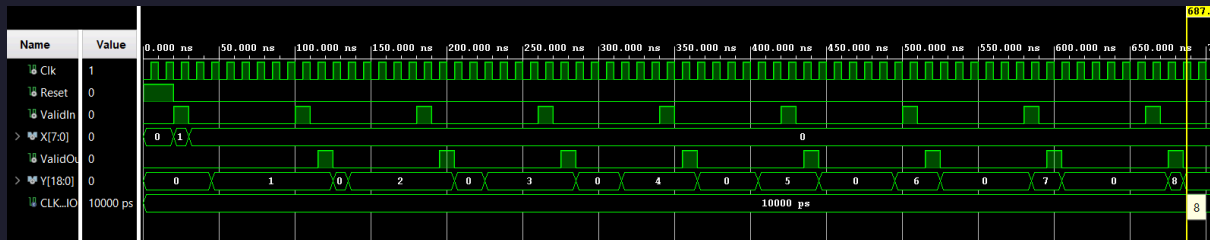


Figure 6: Impulse Response

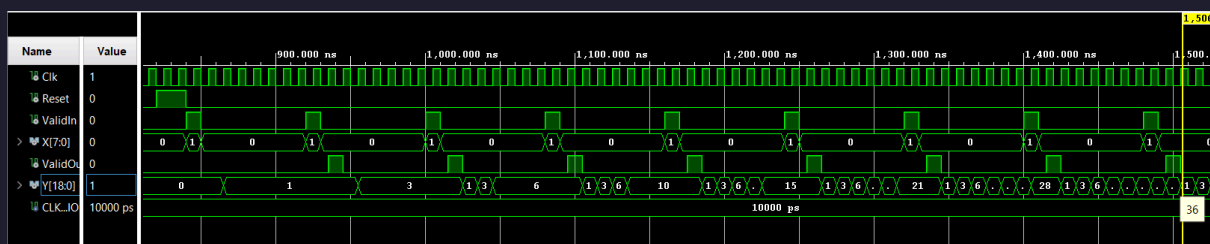


Figure 7: Unit Step Response